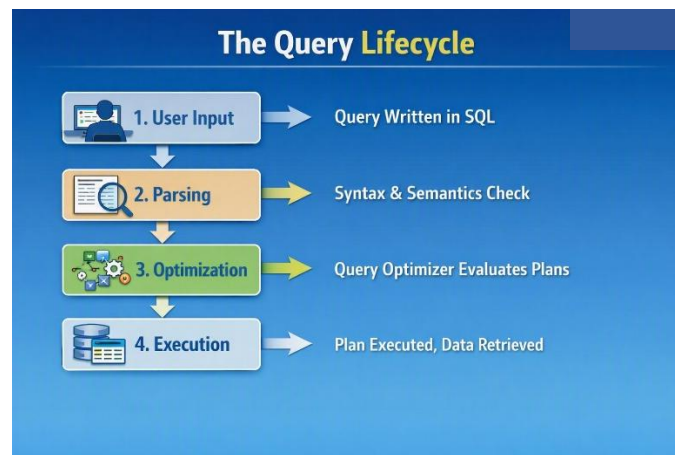


QUERY OPTIMIZATION

Learning Objectives

- Explain the purpose and importance of query optimization in database performance.
- Describe the query life cycle and major components of query optimization.
- Analyze execution plans using EXPLAIN and EXPLAIN ANALYZE.
- Apply indexing, filtering, join optimization, and other techniques to improve SQL query performance.
- Evaluate query performance based on cost, execution time, rows processed, and resource utilization.
- Demonstrate practical skills in optimizing SQL queries using a database management system such as MySQL.

The Query Life Cycle

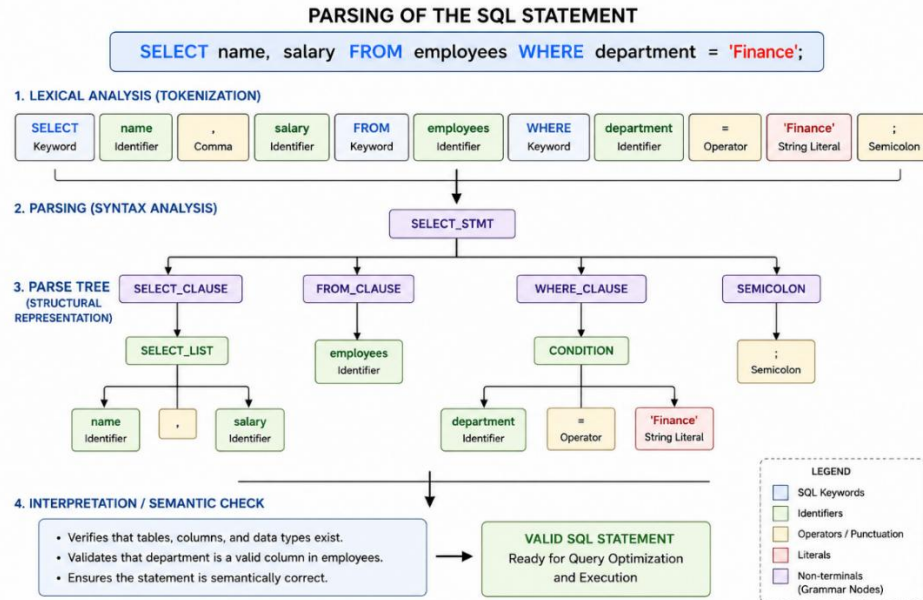


1. User Input

- **Overview:** This is where the process begins. A user writes a query, typically in **SQL (Structured Query Language)**, to request specific data from the database.
- **Purpose:** The query expresses *what* data is needed, not *how* to get it.
- **Example:** SELECT name, salary FROM employees WHERE department = 'Finance';
- **Key Point:** The user's query is declarative — it tells the DBMS what result is desired, leaving the execution details to the system.

2. Parsing

- **Overview:** The DBMS parses the query to check its **syntax** (structure) and **semantics** (meaning).
- **Process:**
 - The **parser** breaks the query into tokens (keywords, identifiers, operators).
 - It verifies that the query follows SQL grammar rules.
 - The **semantic analyzer** ensures that tables, columns, and data types exist and are valid.
- **Output:** A **parse tree** or **query tree**, representing the logical structure of the query.



- **Key Point:** Parsing ensures the query is valid and understandable before execution planning begins.

3. Optimization

- **Overview:** The **query optimizer** evaluates multiple possible execution strategies to find the most efficient one.
- **Process:**
 - Considers different join orders, index usage, and access paths.
 - Estimates the **cost** of each plan (CPU, memory, disk I/O).
 - Selects the plan with the lowest estimated cost.
- **Types of Optimizations:**
 - **Rule-based:** Uses predefined rules. Early database systems often used rule-based optimization, which applied predefined rules to determine execution plans based on query structure.
 - **Cost-based:** Uses statistical data and cost estimation (most modern DBMSs). Modern DBMS typically prioritize cost-based optimization, which evaluates multiple possible execution strategies and estimates the resources required for each one.
- **Key Point:** Optimization is the “brain” of query execution — it determines how to get results fastest and with minimal resource use.

4. Execution

- **Overview:** The DBMS executes the chosen plan to retrieve the requested data.
- **Process:**
 - The **execution engine** interacts with storage, indexes, and buffers.
 - It performs joins, filters, and aggregations as defined in the plan.
 - Results are returned to the user or application.
- **Performance Factors:**

- Disk access speed
- Memory availability
- Query complexity
- **Key Point:** Execution is where the plan becomes action — the database physically retrieves and processes data.

Query Optimization

- Query optimization is the process of finding the most efficient way for a database to execute a query.
- Efficient queries reduce response time, save computational resources, and improve overall system performance.

Why Query Optimization Matters

- Query optimization affects more than single query speed.
- It determines the efficiency of entire data systems, machine learning models, and AI initiatives.
- Core benefits: **scalability** and **resource efficiency**.

Scalability

- Applications depend on fast, consistent database responses.
- Inefficient queries → table scans, sorting, large joins → delays in APIs and analytics.
- As data grows (global datasphere projected at **393.9 zettabytes by 2028**), queries must handle millions or billions of rows.
- Optimization ensures queries remain efficient even as workloads become larger and more complex.

Resource Use

- Every database operation consumes CPU cycles and disk I/O.
- Poorly optimized queries waste resources, increasing costs—especially in cloud environments where usage directly impacts pricing.
- Efficient execution plans reduce unnecessary processing.
- Critical for modern platforms supporting **machine learning, real-time analytics, retrieval-augmented generation (RAG), and AI**.
- Optimization ensures fast, reliable access to large datasets without overspending.

Key Components of Query Optimization

Regardless of the optimization approach used, several technical concepts shape how optimizers evaluate potential execution strategies, including:

1. **Query optimizer** - The query optimizer (called simply the optimizer) is built-in database software that determines the most efficient method for a SQL statement to access requested data.
 - The optimizer attempts to generate the most optimal execution plan for a SQL statement.

- The optimizer chooses the plan with the lowest cost among all considered candidate plans. The optimizer uses available statistics to calculate cost. For a specific query in a given environment, the cost computation accounts for factors of query execution such as I/O, CPU, and communication.
2. **Database statistics** - Optimizers rely heavily on database statistics to estimate how expensive different execution plans will be. Statistics describe key characteristics of stored data, including:
- ✓ Number of rows in each table
 - ✓ Distribution of values within columns
 - ✓ Selectivity of indexed columns
 - ✓ Relationships between tables
 - ✓ Data types of each column
- These statistics allow the optimizer to estimate how many rows a query will return and how much work different execution strategies require. If statistics become outdated or inaccurate, the optimizer may select inefficient execution plans.
3. **Cardinality estimation** - Cardinality estimation refers to predicting how many rows will result from each step in a query. For example, if a query filters rows using WHERE clauses such as:
- WHERE region = 'North America'
- the optimizer must estimate how many records match that filter.
 - These estimates influence several key decisions. The optimizer may use them to determine the order in which tables should be joined, the most efficient join orders, which join algorithms to use or whether an index scan should be used instead of scanning a full table.
4. **Indexes and access paths** - Indexes allow databases to locate specific data more efficiently than scanning entire tables (full table scan). Optimizers use indexes to reduce the amount of work required for data retrieval.
- Choosing the correct access path can significantly reduce the amount of work required to execute a query, particularly when working with large tables.
5. **Join algorithms** - Many queries retrieve data from multiple tables. When this occurs, the optimizer must determine how those tables should be combined. Common join algorithms include:
- **Nested loop joins:** Compares rows from one dataset with rows from another sequentially. This approach can work well when one table is relatively small or when indexes enable fast lookups for inner joins.
 - **Hash joins:** Creates a hash table from one dataset and uses it to efficiently match rows from another dataset. This strategy often works well for large datasets.

- **Merge joins:** Combines rows from two sorted datasets by scanning them simultaneously.

The optimizer selects among these algorithms based on factors such as data size, available indexes and estimated row counts.

Query Optimization Techniques

1. Indexing Strategies

```
mysql> CREATE INDEX idx_region ON country (region);
Query OK, 0 rows affected (0.07 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

Indexes in SQL are special database structures that improve query performance by allowing faster access to data instead of scanning the entire table. They help retrieve records efficiently and enhance overall database performance.

EXPLAIN

MySQL displays information from the optimizer about the statement execution plan. That is, MySQL explains how it would process the statement, including information about how tables are joined and in which order.

EXPLAIN ANALYZE

A profiling tool for your queries that will show you where MySQL spends time on your query and why. It will plan the query, instrument it and execute it while counting rows and measuring time spent at various points in the execution plan. When execution finishes, EXPLAIN ANALYZE will print the plan and the measurements instead of the query result.

Simulation:

Using the table country, with 239 rows/records from the world database (indexing not applied):

EXPLAIN ANALYZE

```
mysql> EXPLAIN ANALYZE SELECT * FROM country WHERE region='Western Europe' ORDER BY name ASC;
+-----+
| EXPLAIN |
+-----+
| -> Sort: country.'Name' (cost=25.6 rows=239) (actual time=0.345..0.348 rows=9 loops=1)
|   -> Filter: (country.Region = 'Western Europe') (cost=25.6 rows=239) (actual time=0.0769..0.314 rows=9 loops=1)
|   -> Table scan on country (cost=25.6 rows=239) (actual time=0.0661..0.279 rows=239 loops=1)
|
+-----+
1 row in set (0.00 sec)
```

After creating index:

```
mysql> EXPLAIN ANALYZE SELECT * FROM country WHERE region='Western Europe' ORDER BY name ASC;
+-----+
| EXPLAIN |
+-----+
| -> Sort: country.'Name' (cost=3.15 rows=9) (actual time=0.134..0.137 rows=9 loops=1)
|   -> Index lookup on country using idx_region (Region='Western Europe'), with index condition: (country.Region = 'Western Europe') (cost=3.15 rows=9) (actual time=0.0724..0.1 rows=9 loops=1)
|
+-----+
1 row in set (0.00 sec)
```

A **table scan** is the easiest and simplest operation that can be performed against a table. It sequentially processes all the rows in the table to determine if they satisfy the selection criteria specified in the query.

In SQL, EXPLAIN ANALYZE executes your query and shows exactly how the database engine handled sorting. **Sort** orders rows based on specified columns. Can be expensive for large datasets.

Query Execution Plan Comparison

Sort	Cost	Rows	Actual Time		Rows	Loops
No index	25.6	239	0.345	0.348	9	1
With index	3.15	9	0.134	0.137	9	1

Query cost is an internal estimate reflecting the resources — CPU, memory, and disk I/O — required for query execution.

The **actual time** values show how long each operation really took: the first number is the time to get the first row, the second is the total time for all rows. Times are usually in milliseconds (ms).

The **rows** value is the planner's estimate of how many rows each operation will process. This helps you understand the scale of data at each step.

The **loops** value shows how many times each operation ran. For example, **loops=1** for the outer scan means it ran once. High loop counts, especially in nested loops, can signal inefficiency if the inner operation is costly or returns many rows.

2. Filtering data earlier

Applying filters earlier in query execution reduces the number of rows that must be processed later in the query. Smaller intermediate results can help speed up query execution. For this reason, queries that apply selective filters early often perform more efficiently.

```
mysql> SELECT * FROM country WHERE region='Western Europe' ORDER BY name ASC;
```

Code	Name	Continent	Region	SurfaceArea	IndepYear	Population	LifeExpectancy	GDP	GDPold	LocalName	GovernmentForm	HeadOfState	Capital	Code2
AUT	Austria	Europe	Western Europe	83859.00	1918	8091800	77.7	211860.00	206025.00	Österreich	Federal Republic	Thomas Klestil	1523	AT
BEL	Belgium	Europe	Western Europe	30518.00	1830	10239000	77.8	249704.00	243948.00	Belgie/Belgique	Constitutional Monarchy, Federation	Albert II	179	BE
FRA	France	Europe	Western Europe	551500.00	843	59229700	78.8	3024285.00	1392040.00	France	Republic	Jacques Chirac	2974	FR
DEU	Germany	Europe	Western Europe	357022.00	1990	82162700	77.5	2113507.00	2102826.00	Deutschland	Federal Republic	Johannes Rau	3068	DE
LIE	Liechtenstein	Europe	Western Europe	160.00	1806	32300	78.8	1119.00	1084.00	Liechtenstein	Constitutional Monarchy	Hans-Adam II	2446	LI
LUX	Luxembourg	Europe	Western Europe	2586.00	1869	435700	77.1	15321.00	15519.00	Luxembourg/Lëtzebuerg	Constitutional Monarchy	Henri	2452	LU
MCO	Monaco	Europe	Western Europe	1.50	1861	34000	78.8	776.00	NULL	Monaco	Constitutional Monarchy	Rainier III	2695	MC
NLD	Netherlands	Europe	Western Europe	41526.00	1581	15564000	78.3	371362.00	360478.00	Nederland	Constitutional Monarchy	Beatrix	5	NL
CHE	Switzerland	Europe	Western Europe	41284.00	1499	7160000	79.6	264078.00	256092.00	Schweiz/Suisse/Svizzera/Svizra	Federation	Adolf Ogi	3248	CH

9 rows in set (0.01 sec)

EXPLAIN

```
mysql> EXPLAIN SELECT * FROM country WHERE region='Western Europe' ORDER BY name ASC;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	country	NULL	ALL	NULL	NULL	NULL	NULL	239	10.00	Using where; Using filesort

1 row in set, 1 warning (0.00 sec)

After creating index:

```
mysql> EXPLAIN SELECT * FROM country WHERE region='Western Europe';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	country	NULL	ref	indx_region	indx_region	104	const	9	100.00	Using index condition

1 row in set, 1 warning (0.00 sec)

The **filtered** column indicates an estimated percentage of table rows that are filtered by the table condition. The maximum value is 100, which means no filtering of rows occurred. Values decreasing from 100 indicate increasing amounts of filtering.

Why a Low "Filtered" Value is a Red Flag

A low filtered percentage (e.g., under 20%) combined with high rows count usually indicates **poor indexing**. It means MySQL is wasting CPU and I/O resources reading a massive amount of data, only to immediately throw most of it away because it doesn't match your WHERE clause.

3. Maintaining database statistics

Because optimizers use statistics to estimate query costs, keeping statistics up to date is essential for maintaining efficient execution plans. Regularly updating statistics ensures that the optimizer has accurate information about data distributions and table sizes.

4. Reducing unnecessary joins

Queries that combine many tables can produce complex queries and equally complex execution plans. When joins are unnecessary or redundant, removing them can significantly reduce execution complexity. In some cases, denormalization can also improve performance by reducing the need for joins, though it may increase storage use and data redundancy.

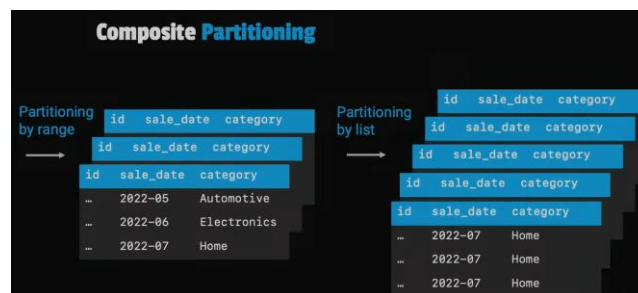
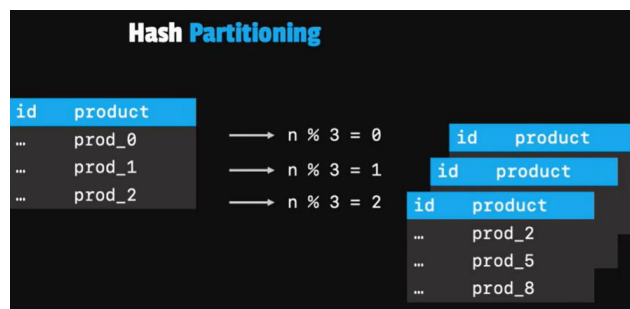
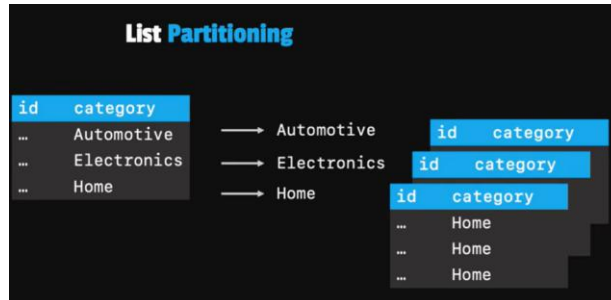
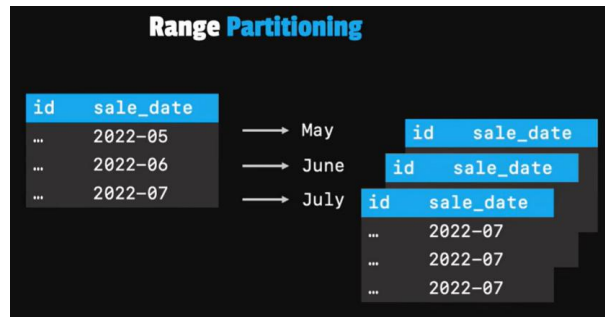
5. Selecting only required columns

Queries that retrieve unnecessary columns increase the amount of data that must be read and processed. Limiting result sets to only the required fields reduce memory usage and disk I/O operations. This small adjustment can noticeably improve performance in large datasets.

6. Partitioning or caching

In some environments, partitioning can help divide very large tables into more manageable segments, while caching can reduce repeated database work for frequently accessed results. These approaches are not universal fixes, but they can complement other optimization strategies.

- Longer response time
- Table bloat - updated and deleted tuples resulted to dead tuple that is need to be cleaned up. Partitioning before vacuum process
- Vacuum reclaims storage space occupied by old or deleted rows and helps maintain database performance. (most commonly associated with PostgreSQL)
- Partitioning methods: List partition, range partition, hash partition, composite partition



Many database platforms also provide built-in tools that help developers and administrators analyze query performance and identify inefficient execution plans.

For example, SQL Server Management Studio (SSMS) can help monitor query performance and identify bottlenecks; MySQL Workbench provides tools for analyzing query plans and optimizing execution; and Oracle SQL Tuning Advisor can generate automated recommendations for improving SQL queries.

References:

- <https://www.ibm.com/think/topics/query-optimization>
- <https://docs.oracle.com/en/database/oracle/oracle-database/18/tgsql/query-optimizer-concepts.html#GUID-6213C0FF-CE34-44AB-97A1-D567820F9419>
- <https://www.sciencedirect.com/topics/computer-science/query-processing>
- <https://dev.mysql.com/doc/refman/8.4/en/using-explain.html>
- <https://www.ibm.com/docs/en/i/7.4.0?topic=table-scan>
- <https://mysqlexplain.com/learning/explain>
- <https://www.geeksforgeeks.org/sql/sql-indexes/>
- hash table https://www.youtube.com/watch?v=59C8c7p_hII
- PostgreSQL Partitioning Tutorial <https://www.youtube.com/watch?v=oJj-pltxBUM>